

ALGORITMOS Y ESTRUCTURAS DE DATOS

Facultad de Ingeniería — UNER

Trabajo Práctico N° 2

Integrantes:

Spiga Edgardo

Micol Reisenstadt

Fecha de entrega: 08/06/2026

Repositorio: <https://github.com/MickRst/AYED2026C1-REISENSTADT-SPIGA>

1. Introducción

Este trabajo práctico aborda tres problemas que aplican estructuras jerárquicas y grafos. En el Problema 1 se implementa un montículo binario de mínimos (MinHeap) para gestionar la sala de emergencias de un hospital, asegurando que siempre se atienda primero al paciente más crítico. En el Problema 2 se construye una base de datos de temperaturas usando un árbol AVL como estructura interna, lo que permite búsquedas y consultas por rango de forma eficiente. En el Problema 3 se modela una red de aldeas como un grafo y se aplica el algoritmo de Prim para encontrar el árbol de expansión mínima.

Los conceptos teóricos aplicados incluyen montículos binarios, árboles AVL con rotaciones de balanceo, grafos no dirigidos ponderados, algoritmo de Prim, complejidad algorítmica, manejo de excepciones, y precondiciones y postcondiciones en docstrings.

2. Desarrollo de los problemas

2.1. Problema N° 1: Sala de Emergencias

A. Análisis y planteo

El problema requiere que siempre se atienda al paciente con mayor riesgo, y ante igual riesgo, al que llegó primero. Esto es exactamente una cola de prioridad. Se eligió un MinHeap genérico porque permite insertar pacientes en $O(\log n)$ y extraer el más prioritario también en $O(\log n)$, siendo más eficiente que mantener una lista ordenada.

La estructura es genérica: no sabe nada de pacientes, simplemente compara elementos usando el operador $<$. La lógica de prioridad se definió en la clase Paciente mediante `__lt__`: primero compara por nivel de riesgo (1=crítico, 2=moderado, 3=bajo) y ante empate compara por número de turno de llegada.

B. Implementación

Se implementaron dos clases: MinHeap en la biblioteca y Paciente en el módulo del proyecto. Las operaciones principales son:

- insertar: agrega al final y sube el elemento (sift-up) — $O(\log n)$.
- extraer_minimo: intercambia raíz con el último, elimina y baja (sift-down) — $O(\log n)$.
- peek: devuelve la raíz sin extraer — $O(1)$.

C. Resultados y pruebas

Se implementaron 15 tests unitarios que cubren el MinHeap, la clase Paciente y la integración de ambos. Todos pasaron correctamente. La simulación mostró que los pacientes críticos siempre se atienden primero y que ante igual riesgo se respeta el orden de llegada.

2.2. Problema N° 2: Temperaturas_DB

A. Análisis y planteo

El científico Kevin Kelvin necesita insertar mediciones frecuentemente y consultar temperaturas por rango de fechas de forma eficiente. Se eligió un árbol AVL porque mantiene el árbol balanceado en todo momento, garantizando $O(\log n)$ para inserción, búsqueda y eliminación independientemente del orden de inserción.

B. Implementación

Se implementaron dos clases: ArbolAVL en la biblioteca y TemperaturaDB en el módulo del proyecto. El AVL implementa rotaciones simples y dobles para mantener el balance. A continuación se presenta el análisis de complejidad de cada método de TemperaturaDB:

Método	Complejidad	Justificación
guardar_temperatura()	$O(\log n)$	Inserción en AVL balanceado
devolver_temperatura()	$O(\log n)$	Búsqueda en AVL balanceado
borrar_temperatura()	$O(\log n)$	Eliminación en AVL balanceado
max_temp_rango()	$O(\log n + k)$	Recorrido in-orden del rango, k nodos visitados
min_temp_rango()	$O(\log n + k)$	Recorrido in-orden del rango, k nodos visitados
temp_extremos_rango()	$O(\log n + k)$	Un solo recorrido in-orden del rango
devolver_temperaturas()	$O(\log n + k)$	Recorrido in-orden más construcción de lista
cantidad_muestras()	$O(1)$	Se retorna el atributo <code>_tamanio</code>
cargar_desde_archivo()	$O(m \log n)$	m líneas del archivo, cada inserción $O(\log n)$

El $O(\log n)$ de las operaciones individuales se debe a que el AVL mantiene la altura acotada en $O(\log n)$ mediante rotaciones. El término k en las consultas por rango representa la cantidad de nodos que caen dentro del rango solicitado.

C. Resultados y pruebas

Se probó la funcionalidad completa con el archivo `muestras.txt` y con datos cargados manualmente. Todas las operaciones funcionaron bien.

2.3. Problema N° 3: Palomas Mensajeras

A. Análisis y planteo

El problema pide encontrar la forma más eficiente de que un mensaje llegue desde Peligros a todas las demás aldeas, minimizando la distancia total recorrida. Este es el problema del árbol de expansión mínima (MST). Se eligió Prim porque parte desde un nodo específico (Peligros), adaptándose mejor al enunciado.

B. Implementación

Se implementaron dos módulos: Grafo y prim. El grafo usa lista de adyacencia (diccionario de diccionarios). El algoritmo retorna un diccionario de padres donde $\text{padre}[v] = (u, \text{peso})$ indica que la aldea v recibe la noticia desde u . La salida muestra en orden alfabético las aldeas, de quién recibe cada una, a quiénes réplica, y la distancia total recorrida.

C. Resultados y pruebas

Se implementaron 16 tests unitarios incluyendo una prueba de integración con el archivo aldeas.txt real. Todos pasaron correctamente.

3. Análisis de resultados específicos / Discusión

Los tres problemas comparten una idea central: elegir la estructura de datos adecuada hace que las operaciones críticas sean eficientes. En el Problema 1, el MinHeap garantiza $O(\log n)$. En el Problema 2, el AVL garantiza $O(\log n)$ independientemente del orden de inserción. En el Problema 3, representar las aldeas como grafo permite aplicar Prim directamente.

Sobre el AVL vs ABB: un árbol binario sin balanceo puede degradarse a una lista enlazada ($O(n)$ por operación) si los datos se insertan en orden. El AVL evita esto con rotaciones, lo que es especialmente importante cuando las fechas podrían insertarse cronológicamente.

4. Declaración de Uso Ético de Inteligencia Artificial

Herramientas utilizadas: Gemini, NotebookLM Pro, Thea

Descripción del uso:

- Asistencia en código (Gemini): Se consultó para identificar errores en la implementación de las rotaciones del AVL y para verificar la lógica del algoritmo de Prim.
- Consulta teórica (Gemini): Se usó para repasar los casos de rotación en árboles AVL (LL, RR, LR, RL) y la diferencia entre Prim y Kruskal.
- Estudio y resúmenes (NotebookLM Pro y Thea): Se utilizaron para estudiar los conceptos teóricos y realizar resúmenes de los contenidos proporcionados por la cátedra, facilitando la comprensión de las estructuras jerárquicas y grafos antes de la implementación.

Declaración de responsabilidad:

Los integrantes del grupo declaramos que hemos revisado y validado personalmente toda la información y código sugerido por las herramientas de IA. Entendemos los conceptos presentados en este informe y asumimos la responsabilidad académica por la integridad y veracidad del contenido.

5. Trabajo en equipo

Organización y roles: Micol Reisenstadt se encargó principalmente de escribir el código de los tres ejercicios, dado que tenía más claros los conceptos teóricos de las estructuras jerárquicas. Spiga Edgardo revisaba lo implementado y corregía detalles que no funcionaban correctamente y adaptaba el código para que fuera coherente con la estructura del repositorio establecida en el TP1. La coordinación se hizo mayormente por llamadas en Discord.

6. Conclusiones

Se cumplieron todos los objetivos del TP: se implementó el MinHeap genérico con su aplicación en la sala de emergencias, la base de datos de temperaturas con árbol AVL, y la solución al problema de las palomas mensajeras con el algoritmo de Prim.

La mayor dificultad fue el árbol AVL, especialmente los cuatro casos de rotación. A diferencia de la LDE del TP1, el AVL requiere pensar en cómo cada operación puede romper el balance y cuál rotación aplicar en cada caso.

Lo más interesante fue ver cómo los tres problemas, aunque distintos, se resuelven con la misma idea: elegir la estructura correcta. Un heap para prioridad, un AVL para búsqueda eficiente, y un grafo con Prim para optimización de rutas.

7. Referencias y Bibliografía

- Miller, B. y Ranum, D. — Problem Solving with Algorithms and Data Structures using Python
- Material de cátedra — Algoritmos y Estructuras de Datos, UNER 2026
- Repositorio del TP — <https://github.com/MickRst/AYED2026C1-REISENSTADT-SPIGA>